

terminal.h4ck1ng.org

Agenda

- 1 Contexto y objetivos
- 2 **Lab 1.** Banco. JWT sin verificación de firma
- 3 **Lab 2.** Foro. Cracking de hashes MD5
- 4 **Lab 3.** Tickets. Desbordamiento de buffer
- 5 Cierre y discusión

Como trabajaremos cada lab

- 1 Intuicion a la idea detras del concepto.
- 2 Introduccion al concepto tecnico.
- 3 Material de referencia (URL, credenciales, indicaciones).
- 4 Tiempo de trabajo individual.
- 5 Solucion comentada y conclusiones.

Lab 1

Hashes, e63a2b4c2, MD5 y otras cosas

¿Cómo hacemos una base de datos?

Al hacer **register**:

- Usuario: jorge
- Clave: vivaEspaña1005
- (La guardamos en la base de datos)

Al hacer **login**:

- Usuario: jorge
- Clave: vivaEspaña1005
- (La buscamos en la base de datos) ¿Coincide?
 - Sí $\Rightarrow$ Loggeamos a jorge
 - No $\Rightarrow$ Clave incorrecta

Intuición 1: Nada

- ¿Cómo hacemos una base de datos segura?

usuario	mail	clave
admin	admin@mail.cl	12345
jorge	jorge@mail.cl	vivaEspaña1005
pangui	pangui@mail.cl	panguiPassword2026

- Si alguien tiene acceso a la base de datos, ¡sabe la clave de todos los usuarios! :(

Intuición 2: Función

- ¿Cómo hacemos una base de datos segura?
- $f(\text{password}) \Rightarrow \text{password123}$
- $f(123) \Rightarrow 123123$

usuario	mail	clave
admin	admin@mail.cl	12345123
jorge	jorge@mail.cl	vivaEspaña1005123
pangui	pangui@mail.cl	panguiPassword2026123

- Notorio, fácil de calcular, determinista

Intuición 3: Función random

- ¿Cómo hacemos una base de datos segura?
- $f(\text{password}) \Rightarrow \text{random}()$

usuario	mail	clave
admin	admin@mail.cl	a3f9c2e
jorge	jorge@mail.cl	7b1d4f8
pangui	pangui@mail.cl	e6c0a52

- ¿Qué pasa cuando un usuario se trata de loggear de nuevo?

¡Idea!

- ¿Y si tenemos una función que retorna un valor que *parece* random, pero es determinista?

Le llamamos **función hash** h :

$$h(\text{clave}) = \text{valor}$$

Por ejemplo:

- $h(\text{vivaEspaña1005}) = 7b1d4f8$
- $h(\text{vivaEspaña1005}) = 7b1d4f8$ (¡siempre lo mismo!)
- $h(\text{vivaEspaña1006}) = c2a91e3$ (cambio mínimo $\Rightarrow$ salida totalmente distinta)

Tres propiedades clave:

- **Determinista:** misma entrada $\Rightarrow$ misma salida
- **Parece random:** la salida no se “parece” a la entrada
- **No se puede invertir:** dado $h(x)$, no podemos recuperar x

Funciones hash: recordatorio

- Una función hash mapea entradas de tamaño *arbitrario* a salidas de tamaño *fijo*:

$$h : \{\text{cualquier texto}\} \longrightarrow \{\text{string de largo fijo}\}$$

- Propiedades clave:
 - **Resistencia a preimagen:** dado $h(x)$, es muy difícil encontrar x .
 - **Resistencia a colisiones:** es muy difícil encontrar $x \neq y$ tal que $h(x) = h(y)$.
- Usos comunes: almacenamiento de contraseñas, integridad de archivos.

```
password = "password123"  
md5(password)  
>> e63a2b4c2e1d3c2e1d31ceba
```

¿Qué problemas pueden haber aquí?

- **MD5** (la más común) fue creada en 1991, su salida es de solo **128 bits**.
- Colisiones prácticas demostradas desde 2004 (Wang et al.).
- Calculable en **miles de millones** de hashes/seg en una GPU moderna.
- Las contraseñas comunes son recuperables con tablas públicas (*rainbow tables*).

Ejemplo

"123456" es una contraseña muy común.

⇒ Busco online si alguien ya calculó su hash.

⇒ Encuentro $h(123456) = \text{e10adc3949ba59abbe56e057f20f883e}$.

⇒ Ahora puedo **revertir** cualquier hash igual a ese.

Lab 1: ForoUnder

[Inicio](#) [Reglas](#) [Tor minor](#) [Crypto](#) 2.847 conectados

 **ForoUnder**
EL FORO LIBRE DE LA RED

Q Buscar en filtraciones, hilos, usuarios...

Mensajes (3) [anon_estudiante](#)

[Inicio](#) [Filtraciones](#) 412 [Hacking](#) 1.8K [Anonimato](#) [Crypto](#) [Mercado](#) [Off-topic](#)

[Inicio](#) > [Filtraciones](#) > [Gobierno y banca](#) > [FILTRACIÓN] Acceso completo a ClaveÚnica — 47K usuarios

[FILTRACIÓN](#) [ESTATAL](#) [HOT](#) [✓ VERIFICADO POR STAFF](#)

[DUMP] Acceso completo a base de datos de ClaveÚnica — 47.892 usuarios + hashes

[jmun_h4ck3r](#) [publicado hace 6 horas](#) [14.231 visitas](#) [87 respuestas](#)


jmun_h4ck3r
• ELITE

Reputación +2.418
Mensajes 1.247
Filtraciones 34
Registrado ene 2022
País 

Publicado 2025-04-30 03:14:09 #1

Qué pasa chavales. Después de tres meses, hoy dropeo lo que les prometí: **acceso completo a la base de ClaveÚnica**. Cuarenta y siete mil ochocientos noventa y dos cuentas, todas con su RUT, email y hash de clave. El endpoint de ClaveÚnica tenía un endpoint expuesto sin rate-limit y yo aprendí en la clase que eso es un problema.

Lo más entretenido es lo que viene después. El **Servicio de Impuestos Injustos** (sí, los mismos que te quitan medio sueldo) confía en ClaveÚnica para todo: renta, F22, boletas, libro de remuneraciones, todo. Si tienes la clave de un usuario, eres ese usuario para el SII.

Alguien sabe si se pueden hackear los hashes para tener las claves?

cu_dump_sample.txt 12 filas · MD5 · sin sat

ID	RUT	EMAIL	PASSMORD_MD
00001	6.482.131-9	jose.contreras@a11.cl	a47f991d2c6

HILOS

Banco de Pangul — JWT vulnerable
1.412 · hace 2 d

[ESTÁS AQUÍ] ClaveÚnica — 47K dump
1.2418 · hace 6 h

REGLAS DEL SUBFORO

- No doxxing de personas naturales.
- Solo dumps de entidades, no de individuos privados.
- Verificación obligatoria por staff antes de pinéar.
- El que vende, fuera. Aquí se libera.
- OPSEC propia. No nos hacemos cargo.

Lab 1: Material de referencia

Acceso

COMMAND	forum
INFO	leak.txt (descargable desde el foro)
LOGIN	Buscar link en el foro

- Intenta loggearte al SII usando la info del foro.
- **Clippy** siempre te puede ayudar!

¿Ganas de más?

Tenemos un juego oculto para seguir practicando.

```
wait
```

Ejecuta este comando para acceder al modo sandbox y hackear un juego completo para llegar a la cima del leaderboard.

¡Que tengas un buen hack!

Lab 1: Solucion y conclusiones

Pasos del exploit

- 1 Descargar leak.txt desde el hilo del foro.
 - 2 Probar cada hash en una rainbow table publica.
 - 3 Hash reversible: 482c811d... → password123.
 - 4 Iniciar sesion en ClaveUnica con ese RUN
-
- MD5 sin sal equivale a contraseñas en texto plano.
 - Reutilizacion de credenciales amplifica el dano de un solo dump.

Lab 2

Login, Usuarios, JSON Web Tokens, y otras cosas

Login: intuición

- Haces login en una página web $\Rightarrow$ ¿cómo sabe la app quién eres en cada request?
- Quizás has escuchado de **cookies**.
- Una cookie no es más que *información sobre ti* guardada en tu navegador.

Ejemplo de cookie

```
{ "user": "jorge", "email": "jorge@mail.cl" }
```

Intuición 1: Nada

- Cada vez que haces una acción $\Rightarrow$ te piden usuario + contraseña de nuevo.
- Seguro, pero **insostenible**: cada click = un login.

Intuición 2: Un ID (o username)

- Al loggearte, el servidor te da un **ID** y tú lo mandas en cada request.
- Por ejemplo:

`cookie = user_id=42`

- Problema: si alguien copia tu ID $\Rightarrow$ **te suplanta**. No hay forma de probar que el ID es realmente tuyo.

¡Idea!

- Necesitamos algo que:
 - te **identifique**, y
 - sea **imposible de falsificar**.
- ¿Te suena familiar? $\Rightarrow$ usemos una función hash con un secreto que solo el servidor conoce.

JWT: definición

- Estándar abierto para transportar *claims* firmados.
- Tres segmentos separados por puntos:

$$\underbrace{\text{header}}_{(1)} . \underbrace{\text{payload}}_{(2)} . \underbrace{\text{signature}}_{(3)}$$

- Ejemplo real:

`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTN0dDkwLCJmYyG3nM`

- Vive en las **cookies** de tu navegador.
- Uso típico: sesiones, autorización entre microservicios.

JWT: estructura

Cómo se construye un JWT:

```
(1) HEADER: { "alg": "HS256", "typ": "JWT" }  
>> JhbGci3Oi1IzMlXM  
  
(2) PAYLOAD: { "user": "jorge", "type": "profe", "admin": false }  
>> eyi0iIxM0iIxMjMONTY3ODkw  
  
(3) SIGNATURE: HASH( base64(header) + "." + base64(payload), secret )  
>> KMUFsIDTnFmyG3nM  
  
JWT = (1).(2).(3)  
      = JhbGci3Oi1IzMlXM.eyi0iIxM0iIxMjMONTY3ODkw.KMUFsIDTnFmyG3nM
```

- El **payload** es legible por cualquiera: es solo Base64, *no* está cifrado.
- La **firma** es la única garantía de integridad.

JWT: el error común

Vulnerabilidad

El servidor **decodifica** el payload
(eyJ0iIxM0iIxMjMONTY3ODkw ⇒ user: jorge)
pero **no valida la firma**.

Consecuencia: cualquier cliente puede modificar el payload y ser aceptado.

Por ejemplo, cambiar:

"admin": false ⇒ "admin": true

"sub": "jorge" ⇒ "sub": "admin"

- Frecuente en bibliotecas mal configuradas y en SDKs propios.

Lab 2: Banco de Pangui

[Sucursales](#) [Cajeros automáticos](#) [Tarifas y tasas](#) [Atención al cliente](#) [Indicadores económicos](#) [Trabaja con nosotros](#) [Sostenibilidad](#)

 **Banco de Pangui**
BANCA EN LÍNEA

¿Qué estás buscando?

[Personas](#) [Pyme y Empresas](#) [Empresas](#) [Banca Privada](#) [Inversiones](#) [Internacional](#)

Su banco de confianza, ahora en línea.

Realice sus transferencias, pagos e inversiones desde cualquier dispositivo, con la seguridad y respaldo que nos caracteriza desde 1893.

[✓ Transferencias inmediatas](#) [✓ Inversión en línea](#) [✓ Sucursal virtual 24/7](#) [✓ Soporte permanente](#)

Banca en Línea **Empresas**

Ingreso clientes
Acceda con su RUT y clave secreta

RUT
12.345.678-9

CLAVE DE ACCESO

 Su sesión está protegida con cifrado TLS 1.3.

INGRESAR

[¿Olvidó su clave?](#) [Primer ingreso](#)

☐ Verifique siempre la dirección antes de ingresar sus datos.

 **Cuenta corriente**
Solicite la suya en línea

 **Tarjetas**
Crédito y débito

 **Inversiones**
Fondos mutuos y DAP

 **Hipotecario**
Cotice su crédito

PRODUCTOS
[Cuentas](#)

ATENCIÓN
[Sucursales](#)

SOBRE EL BANCO
[Quiénes somos](#)

SEGURIDAD
[Recomendaciones](#)

Lab 2: Material de referencia

Acceso

COMMAND	bank
RUT	12.345.678-9
CLAVE	banco2025

- Objetivo: acceder al panel administrativo y recuperar los fondos.
- **Clippy** disponible en todo momento con pistas progresivas.

¿Ganas de más?

Tenemos un juego oculto para seguir practicando.

```
wait
```

Ejecuta este comando para acceder al modo sandbox y hackear un juego completo para llegar a la cima del leaderboard.

¡Que tengas un buen hack!

Lab 2: Solucion y conclusiones

Pasos del exploit

- 1 Login normal; capturar `session_token` en `localStorage`.
 - 2 Decodificar en `jwt.io`; cambiar `admin: false` a `true`.
 - 3 `localStorage.setItem('session_token', ...)` y recargar.
- Nunca confiar en JWT claims sin verificar la firma.
 - Validar tambien `alg`: rechazar `none` y mezclas HS/RS.
 - Rotar secretos; usar bibliotecas auditadas (`jose`, `pyjwt`).

Lab 3

¡Desbordando buffers!

Intuición: ¿dónde viven las variables?

- En Python usan variables, listas y clases sin pensar dónde quedan.
- Pero todo código termina convertido en código de máquina.
- Las máquinas guardan los datos en **memoria**, organizada en **bytes** (1 byte = 8 bits) apilados uno al lado del otro.
- Idea clave: las variables son *vecinas* en memoria.

Pos	Contenido	Significado
7	False	var admin
6	'e'	lista nombre
5	'g'	
4	'r'	
3	'o'	
2	'j'	
1	True	var profesor

¿Cómo puede esto generar problemas?

Escenario: Jorge va a reservar piezas en un hotel.

- El hotel tiene **10 piezas**: 8 normales y 2 premium.
- Las 2 premium ya están ocupadas.
- Costo: \$10/noche normal, \$1000/noche premium.

¿Qué pasa si Jorge **intenta reservar más piezas normales de las que existen?**

Pieza	Huésped
10 (Premium)	Carlos
9 (Premium)	Sofía
8 (Normal)	—
7 (Normal)	—
6 (Normal)	—
5 (Normal)	—
4 (Normal)	—
3 (Normal)	—
2 (Normal)	—
1 (Normal)	—

El truco: pedir 9 cuando solo hay 8

Jorge reserva **9 piezas normales** a nombre de:
Benja, Vicente, Juan, Pedro, Diego,
Pablo, Luis, Ana, Jorge.

El sistema escribe los nombres uno por uno, **sin verificar el límite** de 8.

Resultado: el nombre número 9 (Jorge)
sobrescribió a Sofía.

Jorge ahora figura en una pieza *premium*, pagando 9 piezas normales (\$90 y no \$1000).

Pieza	Huésped
10 (Premium)	Carlos
9 (Premium)	Jorge
8 (Normal)	Ana
7 (Normal)	Luis
6 (Normal)	Pablo
5 (Normal)	Diego
4 (Normal)	Pedro
3 (Normal)	Juan
2 (Normal)	Vicente
1 (Normal)	Benja

En código se ve así

```
# Memoria del hotel: 10 piezas
hotel = [None, None, None, None, None, None, None, None, "Sofia", "Carlos"]
# indices: 0..7 (normal)                                8, 9 (premium)

# Jorge reserva 9 nombres
nombres = ["Benja", "Vicente", "Juan", "Pedro",
           "Diego", "Pablo", "Luis", "Ana",
           "Jorge"]

# El programa NO chequea limites:
for i in range(len(nombres)):
    piezas[i] = nombres[i]

print(piezas[8])
# >> "Jorge"      :)
```


Stack buffer overflow: definición formal

Definición

Escribir **más datos** de los que caben en un buffer del stack $\Rightarrow$ se sobrescribe la memoria *adyacente*.

¿Qué hay en esa memoria adyacente?

- Otras variables (como Sofía en el ejemplo).
- Punteros (direcciones de otras cosas en memoria).
- La **dirección de retorno** de la función.
- Origen histórico: lenguajes sin verificación de límites (C, C++).
- Python normalmente *sobrevive* (chequea índices), pero el bug existe en cualquier lenguaje que toque memoria directamente.

Final del Mundial 2026

The screenshot displays the official ticket portal for the 2026 World Cup Final. The header includes navigation links for 'Tournament', 'Teams', 'News', and 'Help', along with a 'SALES WINDOW: CLOSING SOON' alert. The main content area features the 'WORLD CUP 26 OFFICIAL TICKET PORTAL' logo, a 'Tickets' button, and links for 'Schedule', 'Teams', 'Account', and language options 'EN · ES · PT'. The match is identified as 'KNOCKOUT · FINAL MATCH · 19 JULY 2026'. The teams, CHILE and ESPAÑA, are shown with their respective flags and a 'VS' separator. The match date and time are '19 JUL 2026 · 20:00 LOCAL', with a 'WORLDWIDE BROADCAST' icon. A large countdown timer shows '74 DAYS 01 HOURS 23 MIN 03 SEC'. At the bottom, the 'FINAL · CHI VS ESP · AVAILABLE CATEGORIES' section notes that 'Demand has reached unprecedented levels. Resale prices reflect official secondary market.' It also shows a 'YOUR WALLET \$500.00' and a 'HIGH DEMAND · 1.2M IN QUEUE' status.

Tournament Teams News Help

SALES WINDOW: CLOSING SOON

26 WORLD CUP 26
OFFICIAL TICKET PORTAL

Tickets Schedule Teams Account EN · ES · PT

KNOCKOUT · FINAL MATCH · 19 JULY 2026

THE FINAL.

CHILE VS ESPAÑA

19 JUL 2026 · 20:00 LOCAL WORLDWIDE BROADCAST

74 DAYS 01 HOURS 23 MIN 03 SEC

FINAL · CHI VS ESP · AVAILABLE CATEGORIES

Demand has reached unprecedented levels. Resale prices reflect official secondary market.

YOUR WALLET \$500.00

HIGH DEMAND · 1.2M IN QUEUE

Lab 3: Material de referencia

Acceso

COMMAND final

- Componer el payload en la bandeja y presionar **EXECUTE**.
- El campo custom... acepta texto libre ⇒ ¿podemos hacer overflow?
- **Clippy** te ayuda antes, durante y después del exploit.

¿Ganas de más?

Tenemos un juego oculto para seguir practicando.

```
wait
```

Ejecuta este comando para acceder al modo sandbox y hackear un juego completo para llegar a la cima del leaderboard.

¡Que tengas un buen hack!

Solución y conclusiones

Soluciones exitosas

- SUMMER25 + 9999 $\Rightarrow$ 10 chars total, 2 bytes overflow $\Rightarrow$ total = -5749
- AAAAAAAA + 9999 $\Rightarrow$ 12 chars total, 4 bytes overflow $\Rightarrow$ total = -960M

Matemáticamente, cualquier overflow funciona porque:

$$\text{ASCII mínimo} = 48 \quad ('0') \Rightarrow \text{discount} \geq 48 \Rightarrow \text{total} < 0$$

Lo que se ve aquí es exactamente lo del hotel:

$$\underbrace{\text{AAAAAAA}}_{\text{llena buffer[8]}} + \underbrace{\text{9999}}_{\text{sobrescribe discount[4]}} \Rightarrow \text{total} = \text{price} - \text{discount} < 0$$

- Una variable adyacente al buffer puede ser muy crítica!

Takeaways: ¿qué aprendimos hoy?

- **Lab 1 — Hashes:** Las contraseñas hasheadas sin sal son vulnerables a rainbow tables. MD5 está obsoleto para aplicaciones críticas.
- **Lab 2 — JWT:** Nunca confíes en tokens sin verificar la firma. Los payloads son legibles y editables por cualquiera.
- **Lab 3 — Buffer Overflow:** La memoria es contigua y frágil. Escribir fuera de límites corrompe variables adyacentes.

Principios transversales:

- **Nunca confiar en el input del usuario:** validar siempre longitud, formato y contenido.
- **Principio de menor privilegio:** cada componente debe tener solo los permisos mínimos necesarios.
- **Defensa en profundidad:** múltiples capas de seguridad, no una sola línea de defensa.
- **Las vulnerabilidades son bugs de lógica:** errores pequeños tienen consecuencias enormes.

¿Ganas de más?

Tenemos un juego oculto para seguir practicando.

```
wait
```

Ejecuta este comando para acceder al modo sandbox y hackear un juego completo para llegar a la cima del leaderboard.

¡Que tengas un buen hack!